



SymfonyLive
PARIS 2026

26 - 27 MARS 2026

TALKS EN FRANÇAIS 

#SYMPHONY_LIVE



Jérôme Tamarelle

**"CHIFFREZ VOS DONNÉES AVEC
DOCTRINE, EN RESTANT
RECHERCHABLE"**



@GromNaN

JOIN US!





**VOS DONNÉES ONT ÉTÉ
COMPROMISES !**

Données à caractère personnel

PII – Personally Identifiable Information



1. Identification Directe

- Prénom Nom (Jérôme Tamarelle)
- Numéro de sécurité sociale et de passeport
- Adresse email (jerome@tamarelle.net).
- Réseaux sociaux (@GromNaN)

2. Identification Indirecte

- Identifiants techniques : Adresse IP, Carte de paiement, .
- Données contextuelles : Géolocalisation, plaque d'immatriculation.
- Caractéristiques : "Développeur PHP, qui habite à Rouen et travaille pour MongoDB"



INDUSTRY-FIRST

MongoDB Queryable Encryption

Une **technologie unique** de chiffrement lors de l'utilisation **protège les données sensibles** lors de leur stockage, de leur traitement et de leur consultation, tout au long de leur cycle de vie. Elle permet aux applications de chiffrer les données sensibles **côté client**, de les stocker en toute sécurité dans la base de données MongoDB et d'effectuer des **requêtes complexes directement sur les données chiffrées**.

Algorithmes de chiffrement	Caractéristiques	Date
Code César	Substitution mono-alphabétique	~50 av. J.-C.
Chiffre de Vigenère	Substitution poly-alphabétique	1553
Machine Enigma	Électromécanique à rotors	1918
DES (Data Encryption Standard)	Chiffrement par blocs (56 bits)	1977
RSA (Rivest-Shamir-Adleman)	Asymétrique (Clé publique / privée)	1977
AES-256-GCM (Advanced Encryption Standard)	Symétrique (Blocs - Mode AEAD) Post-quantique	2001 (AES) 2007 (GCM)

Algorithmes de hachage	Usage principal	Date	"Crypto-Safe" ?
CRC32	Détection d'erreurs (Réseau)	1975	Non
MD5	Intégrité de fichiers	1991	Non (Cassé)
SHA-1	Signature numérique	1995	Non (Obsolète)
Bcrypt	Hachage de mots de passe	1999	Oui (Lent par design)
SHA-256	Sécurité standard & Bitcoin	2001	Oui
xxHash (XXH3)	Performance / Big Data	2012 / 2019	Non
Argon2	État de l'art (Mots de passe)	2015	Oui



L'IMAGINATION D'UN
GEEK DE LA CRYPTO:

SON ORDI EST CHIFFRÉ.
CONSTRUISONS UN CLUSTER À UN
MILLION DE DOLLARS POUR
LE CRACKER.

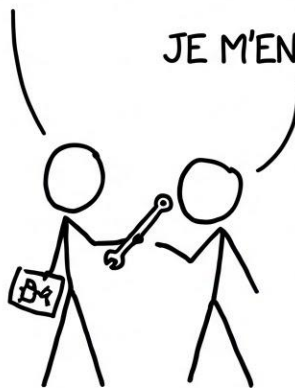
INUTILE ! C'EST DU
RSA 4096 BITS!

MINCE ! NOTRE
PLAN DIABOLIQUE
EST DÉJOUÉ!

CE QUI SE PASSERAIT
RÉELLEMENT:

SON ORDI EST CHIFFRÉ.
DROGUONS-LE ET FRAPPONS-LE
AVEC CETTE CLÉ À MOLETTE
À 5 \$ JUSQU'À CE QU'IL NOUS
DONNE LE MOT DE PASSE.

JE M'EN OCCUPE.



Chiffrement

```
function encrypt(string $plaintext, string $dek): string
{
    $iv = // ...

    $ciphertext = openssl_encrypt($plaintext, 'aes-256-cbc', $dek, OPENSSL_RAW_DATA, $iv);
    return $iv . $ciphertext;
}
```


Chiffrement aléatoire

Le résultat de l'opération de chiffrement change aléatoirement

```
function encrypt(string $plaintext, string $dek): string
{
    $iv = random_bytes(16);

    $ciphertext = openssl_encrypt($plaintext, 'aes-256-gcm', $dek, OPENSSL_RAW_DATA, $iv);
    return $iv . $ciphertext;
}
```

Chiffrement déterministe

Le résultat de l'opération de chiffrement est toujours identique pour une valeur donnée

```
function encrypt(string $plaintext, string $dek): string
{
    $hash = hash('sha512', $plaintext, true);
    $iv = substr($hash, 0, 16);

    $ciphertext = openssl_encrypt($plaintext, 'aes-256-gcm', $dek, OPENSSL_RAW_DATA, $iv);
    return $iv . $ciphertext;
}
```

La valeur du hash peut être comparé avec d'autres bases de données !

Chiffrement déterministe

Le résultat de l'opération de chiffrement est toujours identique pour une valeur donnée

```
function encrypt(string $plaintext, string $dek): string
{
    $hmac = hash_hmac('sha512', $plaintext, $dek);
    $iv = substr($hmac, 0, 16);

    $ciphertext = openssl_encrypt($plaintext, 'aes-256-gcm', $dek, OPENSSL_RAW_DATA, $iv);

    return $iv . $ciphertext;
}
```

(MongoDB utilise une clé dérivée différente pour le hash et le chiffrement)

Déchiffrement

Le résultat de l'opération de chiffrement est toujours identique pour une valeur donnée

```
public function decrypt(string $payload, string $dek): string
{
    // Split the payload into IV and ciphertext.
    $iv = substr($payload, 0, 16);
    $ciphertext = substr($payload, 16);

    $plaintext = openssl_decrypt($ciphertext, 'aes-256-gcm', $dek, \OPENSSL_RAW_DATA, $iv);

    if ($plaintext === false) {
        throw new RuntimeException('Decryption failed: ' . openssl_error_string());
    }

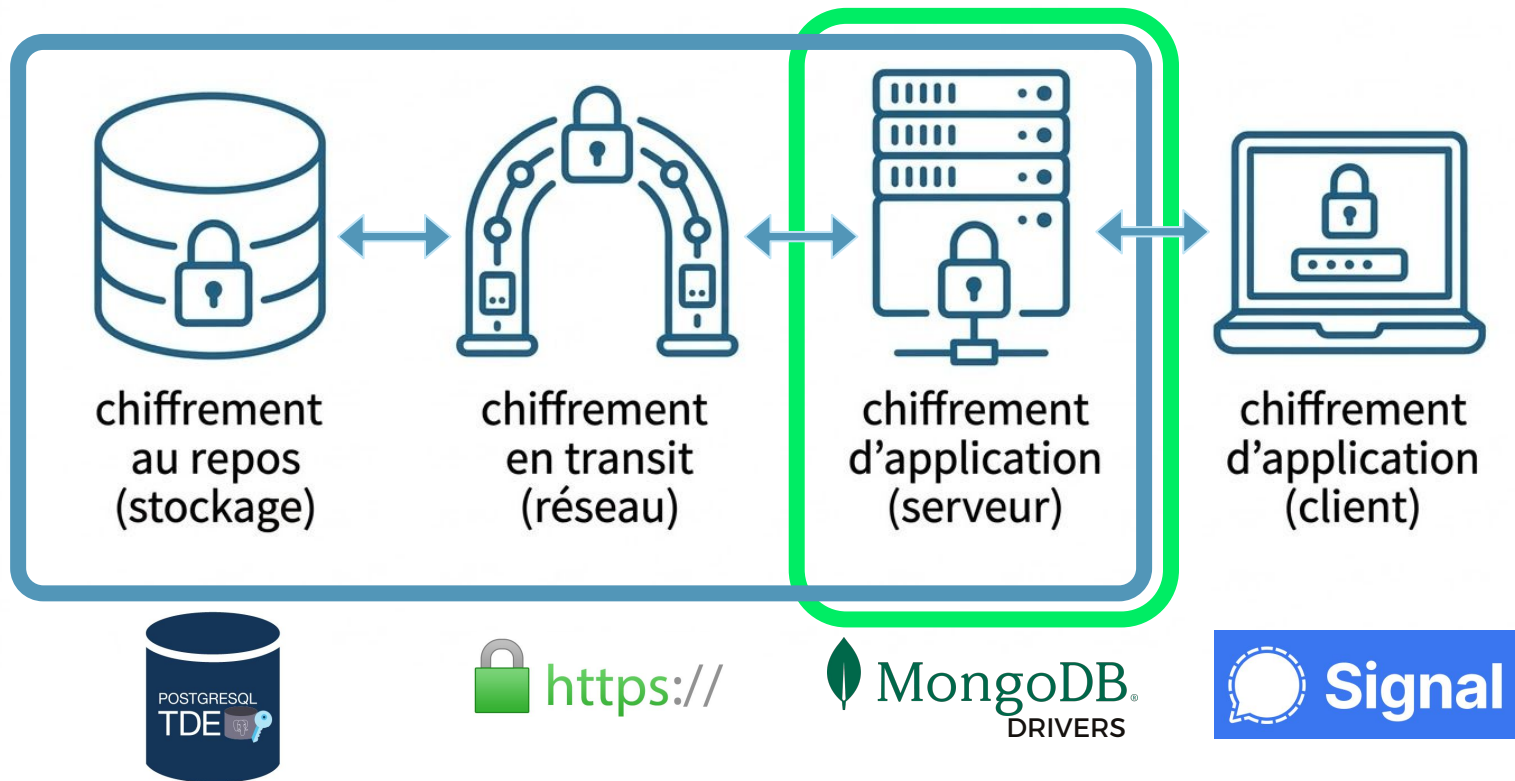
    return $plaintext;
}
```

Pourquoi ne jamais utiliser un hash sans secret ?

Si on fait juste `$iv = substr(hash('sha512', $email), 0, 16)`, voici ce qui se passe :

1. Un pirate récupère la DB.
2. Il voit une ligne avec un email chiffré.
3. Il se demande : *"Est-ce que l'utilisateur 'admin@societe.fr' est dans cette base ?"*.
4. Il calcule `hash('sha512', 'admin@societe.fr')` sur sa propre machine.
5. S'il trouve le même début de chaîne que ton IV en base, **BINGO**, il a identifié la victime sans même avoir besoin de déchiffrer.

Règle d'or : Un hash sans clé pour des données identifiables (PII) est une faille de confidentialité.



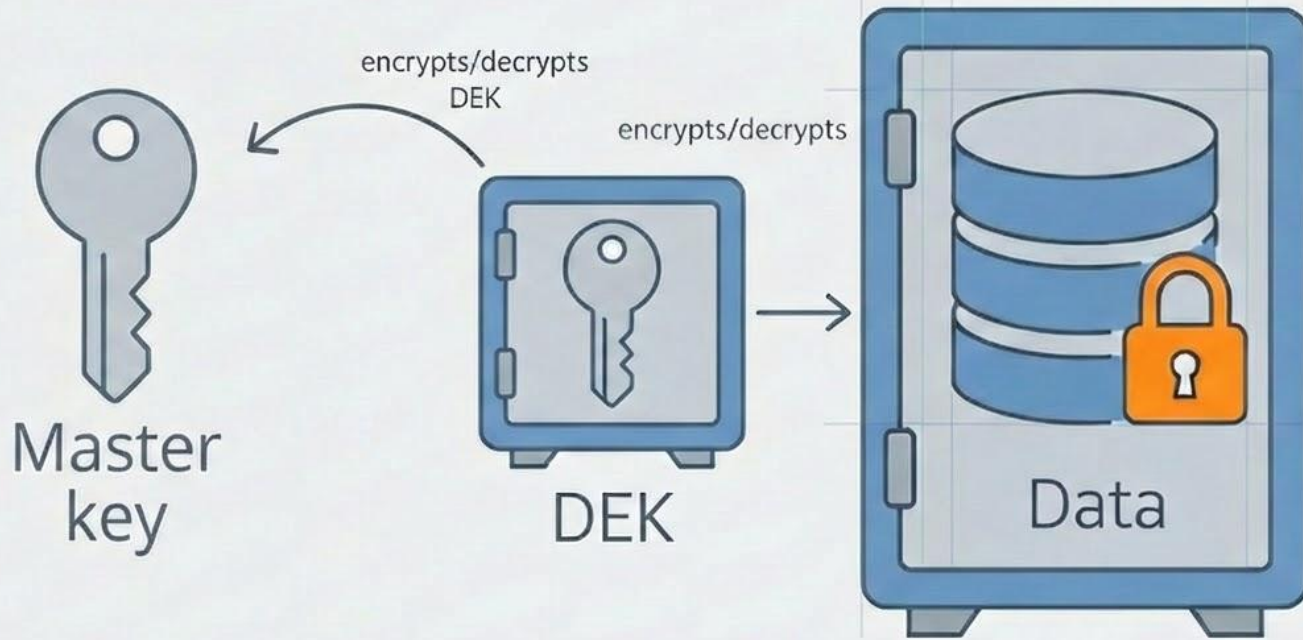


Data

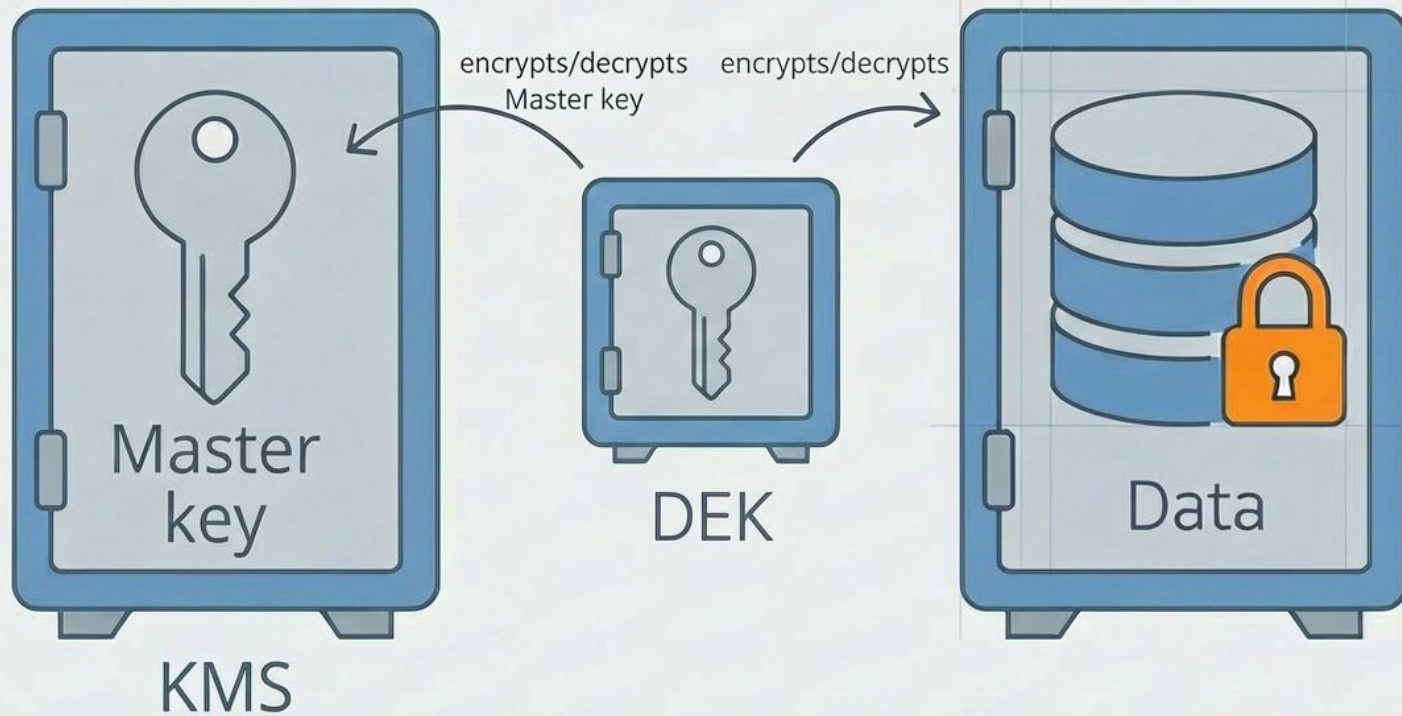
1. La donnée sensible nécessite une protection



1. La donnée sensible nécessite une protection
2. Cette donnée est chiffrée avec une DEK (Data Encryption Key)



1. La donnée sensible nécessite une protection
2. Cette donnée est chiffrée avec une DEK (Data Encryption Key)
3. La DEK est sécurisée par une Master Key



1. La donnée sensible nécessite une protection
2. Cette donnée est chiffrée avec une DEK (Data Encryption Key)
3. La DEK est sécurisée par une Master Key
4. Le KMS (Key Management System) protège la Master Key



Choisissez un Key Management System (KMS)



tmpfs:///secrets/master.key



Type de chiffrement

Déterministe

- + Recherche d'égalité
- Vulnérable à l'analyse de fréquence

Aléatoire

- + Sécurité maximale
- Impossible à filtrer

Chiffrer les champs en BDD

```
#[ORM\Entity]
class User
{
    #[ORM\Id]
    #[ORM\Column]
    public ?int $id = null;

    // Deterministic encryption of the email enables unique indexing
    #[ORM\Column(length: 512, type: Types::STRING, unique: true)]
    public string $email;

    // Random encryption prevents correlating identical first names
    #[ORM\Column(length: 512, type: Types::BINARY)]
    public string $firstName;

    #[ORM\Column(length: 512, type: Types::BINARY)]
    public string $lastName;
}
```

Chiffrer les champs en BDD

```
$user = new User();  
$user→email = $encryptor→encryptDeterministic('fabien@symfony.com', $dek);  
$user→firstName = $encryptor→encryptRandom('Fabien', $dek);  
$user→lastName = $encryptor→encryptRandom('Potencier', $dek);
```

```
$encryptor→decrypt($user→email, $dek); // fabien.potencier@symfony.com  
$encryptor→decrypt($user→firstName, $dek); // Fabien  
$encryptor→decrypt($user→email, $dek); // Potencier
```


Est-ce suffisamment aléatoire ?

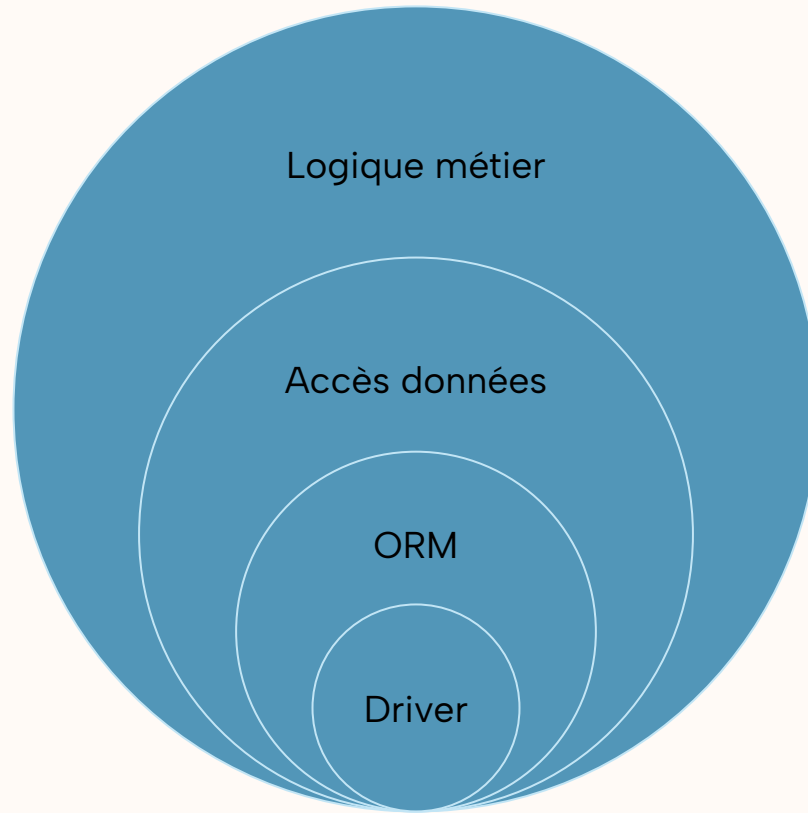
`random_bytes()`

`openssl_random_pseudo_bytes()`

`sodium_randombytes_buf()`

La randomisation générée par cette fonction est adaptée à toutes les applications, y compris la génération de secrets à long terme, tels que des clés de chiffrement.



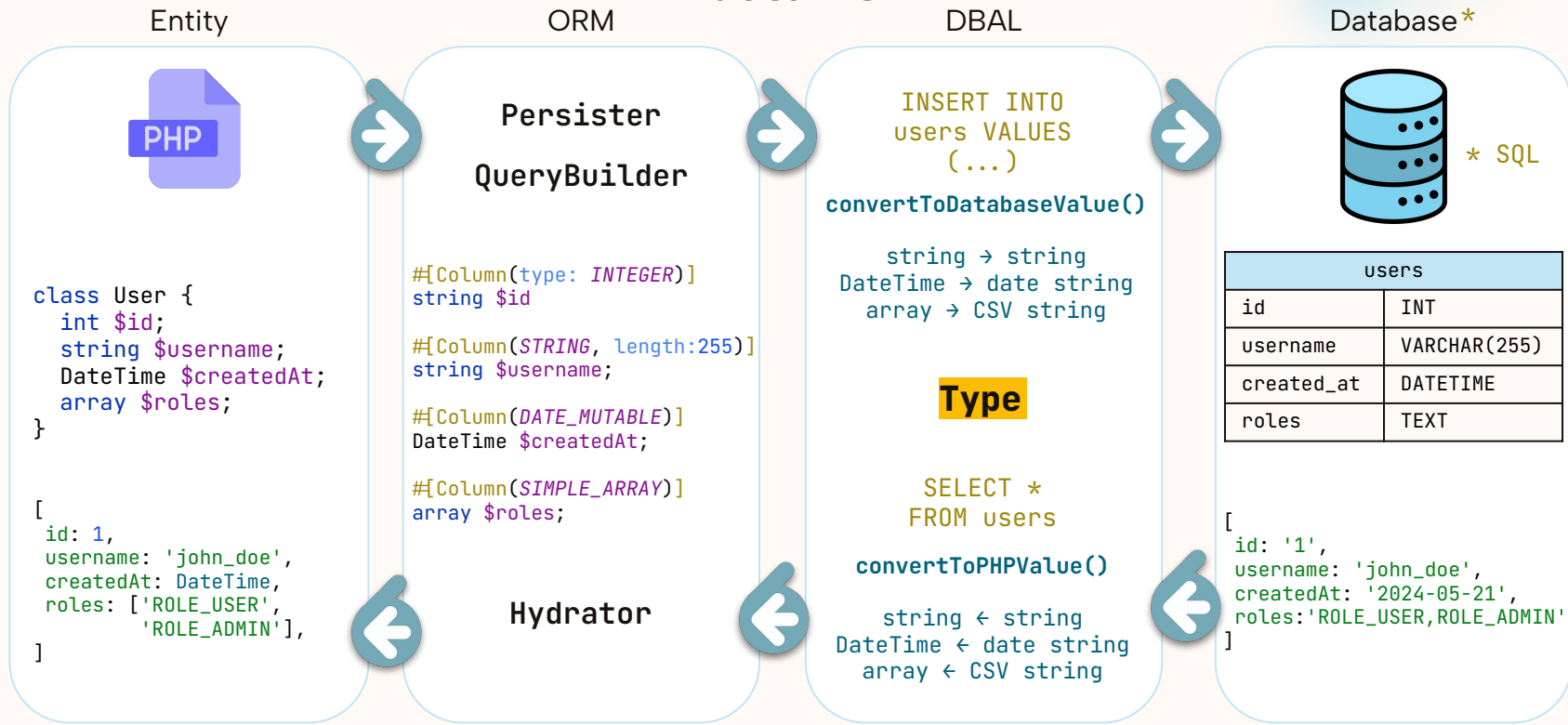


LA Clé de chiffrement

Chiffrement par champs avec l'ORM



doctrine



Création d'un Type personnalisé

```
final readonly class EncryptedType extends \Doctrine\DBAL\Types\Type
{
    public function convertToDatabaseValue(mixed $value, AbstractPlatform $platform): mixed
    {
        // ...
    }

    public function convertToPHPValue(mixed $value, AbstractPlatform $platform): mixed
    {
        // ...
    }
}
```

Création d'un Type personnalisé

```
public function convertToDatabaseValue(mixed $value, AbstractPlatform $platform): mixed
{
    $plaintext = $this->parentType->convertToDatabaseValue($value, $platform);

    if ($plaintext === null) {
        return null;
    }

    $cyphertext = $this->encryptor->encryptRandom($this->dekId, $plaintext);

    return $cyphertext;
}
```

Création d'un Type personnalisé

```
public function convertToPHPValue(mixed $value, AbstractPlatform $platform): mixed
{
    if (!$value) {
        return null;
    }

    $plaintext = $this->encryptor->decrypt($this->dekId, $value);

    return $this->parentType->convertToPHPValue($plaintext, $platform);
}
```


Création d'un Type personnalisé

```
public function getBindingType(): ParameterType
{
    return ParameterType::BINARY;
}
```

```
public function getSQLDeclaration(array $column, AbstractPlatform $platform): string
{
    return $platform->getBinaryTypeDeclarationSQL($column);
}
```

Création d'un Type personnalisé

```
use Doctrine\DBAL\Types\Type;
use App\Encryption\DekEncryptionService;

final readonly class EncryptedType extends Type
{
    public function __construct(
        private Type $parentType,
        private DekEncryptionService $dekEncryptionService,
        private string $dekId,
    ) {}
}
```

Remove final constructor for Type #6705

[Code](#)

Merged greg0ire merged 4 commits into doctrine:4.3.x from ruudk:ruudk/2025/1/remove-final-type-constructor on Feb 20, 2025

Conversation 31

Commits 4

Checks 66

Files changed 4

+89 -12



ruudk commented on Jan 13, 2025 • edited

Contributor ...

Q	A
Type	improvement
Fixed issues	#6704

Summary

This allows people to define a constructor when creating custom `Doctrine\DBAL\Types\Type` implementations.

They can be registered by passing an instance to `Type::addType`.

When a class string is passed, and the Type cannot be instantiated, an helpful error message will be thrown.



ruudk mentioned this pull request on Jan 13, 2025

Documentation about Custom Mapping Types with custom constructor is invalid [#6704](#)

Closed

Reviewers

- greg0ire ✓
- SenseException ✓
- derrabus ●
- +1 more reviewer ^
- GromNaN ✓

Assignees

No one assigned

Labels

[Improvement](#)

Projects

None yet

Milestone

4.3.0

Création d'un Type personnalisé

```
$type = new EncryptedType(  
    Type::getTypeRegistry()→get('string'),  
    $container→get(Encryptor::class),  
    'my-dek-id',  
);  
  
Type::getTypeRegistry()→register('user_name_encrypted', $type);
```

Création d'un Type personnalisé

```
#[ORM\Entity]

class User
{
    #[ORM\Id, ORM\GeneratedValue, ORM\Column]
    public ?int $id = null;

    #[ORM\Column(type: 'user_email_encrypted', unique: true)]
    public string $email;

    #[ORM\Column(type: 'user_name_encrypted')]
    public string $name;
}
```

Les types sont utilisés lorsque les champs sont identifiés

- ✓ Hydratation depuis la base de données vers les classe entitée
- ✓ Persistance des entitées vers la base de données
- ✓ Requêtes utilisant le query builder
- ✗ Requêtes DQL ou SQL
- ✗ Résultats bruts de la base de données

Limitations

1. Le diff pour la persistance est calculé à partir de la représentation “database”. Celle-ci change à chaque chiffrement aléatoire. Il y a donc systématiquement des changements détectés sur les entités.
2. Les types agissent sur la valeur de chaque champs individuellement. Impossible de stocker la valeur d’une propriétés dans plusieurs colonnes.

Recherche sur données cryptées

```
#[ORM\Entity]
class User
{

    #[ORM\Column(type: 'user_email_encrypted')]
    public string $email;

    #[ORM\Column(type: Types::STRING, length: 64)]
    public string $emailHash;

    #[ORM\Column(type: Types::STRING, length: 64)]
    public string $emailDomainHash;

}
```


Recherche sur données cryptées

```
#[AsDoctrineListener(event: \Doctrine\ORM\Events::prePersist)]
#[AsDoctrineListener(event: \Doctrine\ORM\Events::preUpdate)]

final class UserEmailHashListener
{
    public function prePersist(PrePersistEventArgs $args): void
    {
        $this→applyHashes($args→getObject());
    }

    public function preUpdate(PreUpdateEventArgs $args): void
    {
        $this→applyHashes($args→getObject());

        $em = $args→getObjectManager();
        $uow = $em→getUnitOfWork();
        $metadata = $em→getClassMetadata(UserEmailEncrypted::class);
        $uow→recomputeSingleEntityChangeSet($metadata, $args→getObject());
    }
}
```

Recherche sur données cryptées

```
final class UserEmailHashListener
{
    public function __construct(private readonly string $hmacKey) {}

    private function applyHashes(UserEmailEncrypted $entity): void
    {
        $normalizedEmail = strtolower(trim($entity->email));
        $entity->emailHash = hash_hmac('sha256', $normalizedEmail, $this->hmacKey);

        $domain = explode('@', $normalizedEmail, 2)[1] ?? '';
        $entity->emailDomainHash = hash_hmac('sha256', $domain, $this->hmacKey);
    }
}
```

Recherche sur données cryptées

```
final class UserRepository extends ServiceEntityRepository
{
    public function findByEmailDomain(string $domain): array
    {
        $normalizedDomain = strtolower(trim($domain));
        $domainHash = hash_hmac('sha256', $normalizedDomain, $this->hmacKey);

        return $this->createQueryBuilder('u')
            ->andWhere('u.emailDomainHash = :domainHash')
            ->setParameter('domainHash', $domainHash)
            ->getQuery()
            ->getResult();
    }
}
```

Recherche sur données cryptées

```
#[ORM\Entity]
class User
{
    #[ORM\Column(type: 'user_email_encrypted')]
    public string $email;

    /** @var list<string> Liste de tags de recherche hachés */
    #[ORM\Column(type: Types::SIMPLE_ARRAY)]
    public array $searchTags;
}
```



<https://jerome.tamarelle.net/slides/range-encryption-demo.html>

ÉTAPE 1

ÉTAPE 2

ÉTAPE 3

ÉTAPE 4

ÉTAPE 5

ÉTAPE 6

ÉTAPE 7

ÉTAPE 8

Résultat

L'item est trouvé via [13-14], mais plus l'intervalle est étendu, plus la requête est coûteuse

1. Sans arbre - 4/4

Valeur 13

Requête [9, 22]

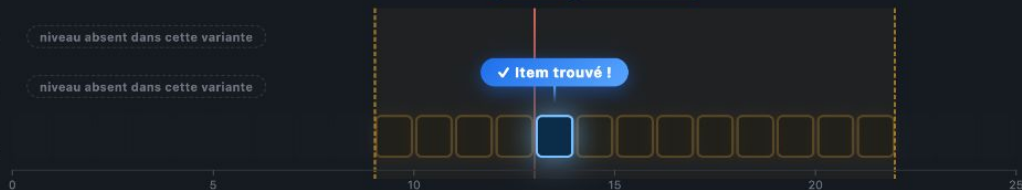
N0

niveau absent dans cette variante

N1

niveau absent dans cette variante

N2



Segment (inactif)

Segment stocké (valeur 13)

Segment recherché (requête [9-22])

Segment commun (intersection)

Niveau absent dans la variante

Segment commun [13-14]

Coût requête 13 tags

PostgreSQL — résultat

Chiffrement désactivé

id	sensitive_number	tags
----	------------------	------

1	13	{"N2: [13-14]"}
---	----	-----------------

→ La ligne est retenue car le tableau `tags` intersecte

→ le tableau des segments demandés par la requête.

L'item est bien retrouvé, ici grâce au segment exact [13-14].

La démonstration marche, mais elle montre aussi la limite du modèle sans arbre : la requête est *verbeuse* et envoie beaucoup de tags.

On peut maintenant introduire l'arbre pour conserver le même résultat, avec une requête *beaucoup plus compacte*.



SymfonyLive
PARIS 2026

★ martin-georgiev/postgresql-for-doctrine

↓ `composer require martin-georgiev/postgresql-for-doctrine`

Extends Doctrine with native PostgreSQL support for arrays, JSONB, ranges, PostGIS geometries, text search, ltree, uuid, and 100+ PostgreSQL-specific functions.

Maintainers



Package info

[github.com/martin-georgiev/postgres...](https://github.com/martin-georgiev/postgresql-for-doctrine)
`pkg:composer/martin-georgiev/postgr...`

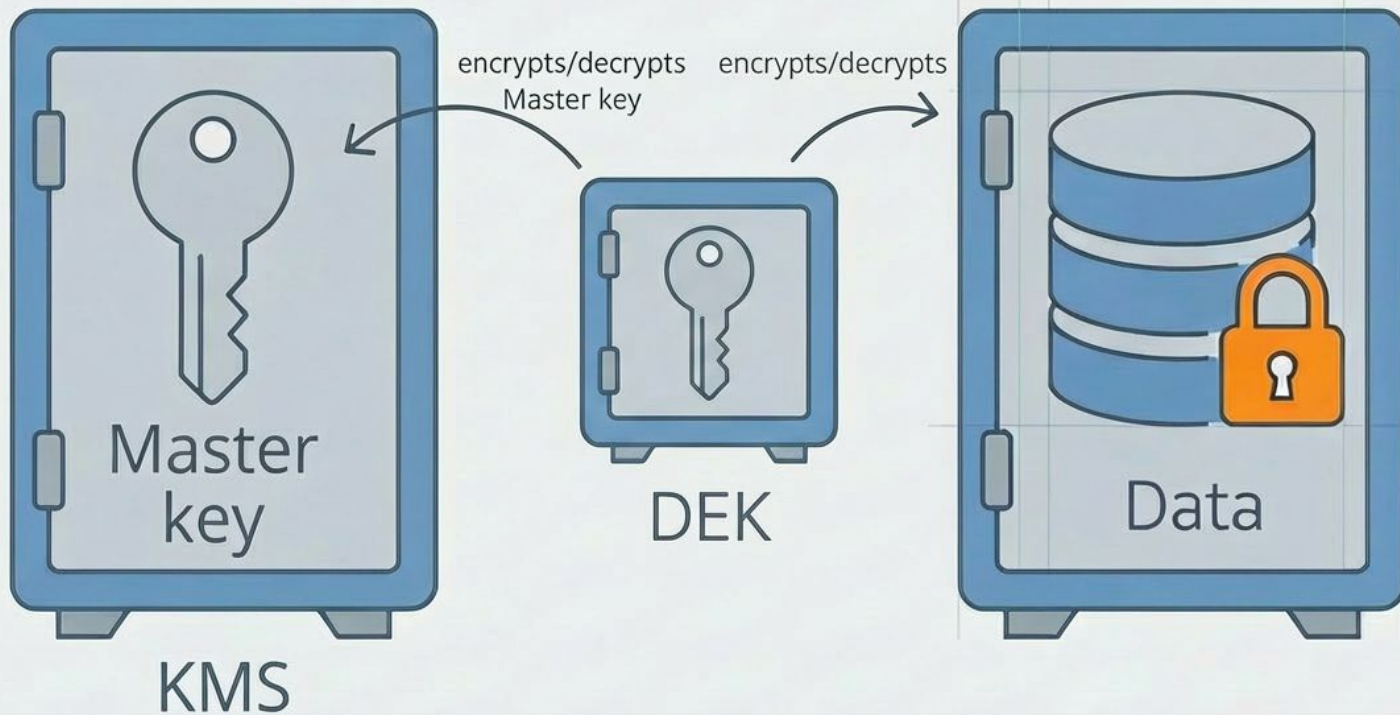
Fund package maintenance!

 [martin-georgiev](#)
 [github.com/sponsors/martin-georg...](https://github.com/sponsors/martin-georgiev)

Statistics

Installs:	5 261 907
Dependents:	5
Suggesters:	1
Stars:	447
Open Issues:	10

Rotation de clé



Rotation de clé = rechiffrer avec la nouvelle clé

Clé à remplacer	Information chiffrée	Volume	Durée	Fréquence
Master Key	Data Encryption Key	Faible	Rapide	6 à 24 mois
Data Encryption Key	Donnée métier	Énorme	Très long	Si besoin

Rotation de clé = rechiffrer avec la nouvelle clé

Les données chiffrées sont toujours stockées avec l'identifiant de leur clé de chiffrement

- DEK :
 `id: <id>,`
 `masterKey: {provider: "aws", key: "<arn>", ...},`
 `keyMaterial: <binary>`
- Donnée : `Binary(<dekId>.<iv>.<donnée chiffrée>)`

#[Doctrine\ODM\MongoDB\Mapping\Attribute\Encrypt]

Doctrine MongoDB ODM

Support natif du chiffrement par champs, requêtable

```
use Doctrine\ODM\MongoDB\Mapping\Attribute as ODM;
use Doctrine\ODM\MongoDB\Mapping\EncryptQuery;

#[ODM\Document]
class UserEncrypted
{
    #[ODM\Id]
    public ?int $id = null;

    #[ODM\Field, ODM\Encrypt]
    public string $name;

    #[ODM\Field, ODM\Encrypt(queryType: EncryptQuery::Equality)]
    public string $email;
```

Doctrine MongoDB ODM

Support natif du chiffrement par champs, requêtable

```
#[ODM\Field]
#[ODM\Encrypt(
    queryType: EncryptQuery::Range,
    min: new \DateTimeImmutable('1900-01-01'),
    max: new \DateTimeImmutable('2050-01-01'),
)]
public \DateTimeImmutable $birthday;
```

Pourquoi choisir MongoDB ?



- Implémentation cross-driver (indépendant du langage et du framework)
- Stockage optimisé pour les performances de stockage et recherche
- Sécurité accrue avec validation du schéma côté serveur
- Support de l'éditeur
- Autres : modèle document, index Lucene, vecteurs, auto-embedding,...

Quoi de neuf dans MongoDB 8.2 ?



SUPPORT EXISTANT

Equality

Récupérer un enregistrement pour un client avec l'adresse e-mail chiffrée exacte "alex@example.com"

SUPPORT EXISTANT

Range

Trouver tous les salaires chiffrés entre 50 000 € et 80 000 €

NOUVEAU

Prefix

Trouver tous les clients dont le nom de famille commence par "Mac"

NOUVEAU

Suffix

Correspondre aux quatre derniers chiffres d'un numéro de sécurité sociale chiffré, comme "-1234"

NOUVEAU

Substring

Rechercher un mot-clé comme "ingénieur" dans un titre de poste ou une description chiffrée en texte libre

Pourquoi choisir Doctrine + Postgres ?



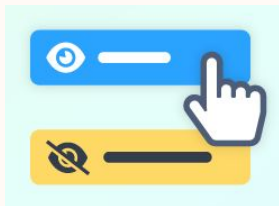
- Implémentation totalement open-source et adaptable
- Ajout progressif de chiffrement par champs dans une application existante
- Si c'est la technologie maîtrisée par les équipes

Le chiffrement ne fait pas tout : les données fuient via les interfaces utilisateur

Restriction d'accès
au strict nécessaire



Click-to-reveal



Rate Limiter
(limite de volume)



Surveillance et alertes
sur l'usage abusif

